Introduction to the ZFS Filesystem

Benedict Reuschling

January 13, 2025



Introduction to the ZFS Filesystem

These slides provide an overview of the ZFS filesystem. In addition to exploring the fundamental design concepts, we will delve into features of interest to users. The presentation will guide you through the creation and management of a storage pool, configuration of ZFS parameters, compression, serialization of pool data, self-healing in case of failure, and deduplication.
The examples aim to be distribution-independent, utilizing FreeBSD device names, which are interchangeable with those used by other Unix distributions. Given that ZFS encompasses more features than covered in this slide deck, our focus will be on the most significant features for end users. OpenZFS[1], arguably the most widely used and actively maintained implementation, will be used as our reference.

[1] `http://open-zfs.org/wiki/Main_Page`

Overview

1. Introduction
2. Problems with Today's Filesystems
3. Features of ZFS
   - Simple Administration
   - Quota and Reservations
   - Snapshots
     - Clones
     - Boot Environments
   - Self-Healing Data
   - Compression
   - Deduplication
   - ZFS Volumes
   - ZFS Serialization

Introduction

ZFS was developed at Sun Microsystems, later acquired by Oracle, as a new filesystem and volume manager, including support for RAID. It was released as open source before the Oracle takeover. After the acquisition, many former ZFS developers left Oracle, taking the latest open-source version of ZFS, and founded the OpenZFS project.
ZFS was initially designed for use in storage systems and servers handling large amounts of data. However, its features also make it suitable for Unix desktops at home with comparatively smaller storage requirements. ZFS's versatility makes it appealing for various use cases, both large and small.
During ZFS development, the designers focused on the following key points:

- Data integrity - detect and correct data corruption and errors
- Storage capacity - the first 128 bit filesystem
- Simple administration - 2 commands are enough to manage the storage
- High Performance

Problems with Today's Filesystems

Most filesystems are still rooted in assumptions from 20 years ago, even as the development of more advanced disk drives, featuring faster access and higher capacities, has progressed significantly. The advent of SSDs (Solid State Disks) necessitates changes in filesystems to align with the storage requirements of contemporary systems.
Some drawbacks from current filesystems are:

Silent data corruption: Errors can occur at various points in the I/O chain before data is stored. Unfortunately, these errors often go unnoticed, making it challenging to detect corrupted data when accessed laterby which point it's already too late. Issues may arise in disks, controllers, cables, drivers, firmware, or RAM without error-correcting code (ECC), preventing proper data storage without notifying the user or operating system.

Problems with Today's Filesystems

Bad administrative tools: Data losses can occur due to the complexities involved in the administration of data by users or administrators. A clear understanding of terminology such as labels, partitions, volumes, and configuration files like `/etc/fstab` is crucial. Proper configuration of these elements is necessary to effectively utilize storage resources. However, the multitude of administrative commands and parameters can make the process challenging and increase the potential for errors instead of simplifying it.

Limits: Volume- and file sizes, limits in upper number of files, files per directory, number of snapshots and other details are always limited to a specific number.

Slow processing: Locking, fixed block sized, slow writes of non-contiguous files slow down I/O performance.

In the server space these factors play an important role to make data available, efficiently, and securely. **ZFS** implements fixes to the problems mentioned above.

Overview

1. Introduction
2. Problems with Today's Filesystems
3. Features of ZFS
   - Simple Administration
   - Quota and Reservations
   - Snapshots
     - Clones
     - Boot Environments
   - Self-Healing Data
   - Compression
   - Deduplication
   - ZFS Volumes
   - ZFS Serialization

Features of ZFS

Developed from scratch, ZFS fits today's requirements for performance and data integrity. Putting a special focus during development on:

- Pool-based storage - a managed storage pool on which the whole storage logic and administration resides
- Data integrity from beginning to the end - verifying and correcting data if necessary using checksums at each point of the storage chain (from RAM down to the disks)
- Transactional operations - this model, known from databases, allows constant consistency as well as high performance
- Copy-on-write (COW) - existing data is never overwritten, but stored at a new location. Due to that, the data is always consistent with no need for `fsck`.

ZFS Requirements

ZFS has somewhat higher system requirements compared to other filesystems. Main memory plays a crucial role in ZFS, as a significant portion of data is pre-sorted in RAM and processed before being orderly written to disk. It is not recommended to use ZFS on 32-bit systems for production workloads due to the limitations of this architecture, which supports up to 4 GB of RAM. In contrast, 64-bit systems should allocate and reserve at least $>=$ 4 GB of RAM for ZFS. Despite the higher system requirements, the performance benefits are well worth it, given that the filesystem, responsible for managing tertiary storage (disks), is typically the slowest part of a computer system.

Deduplication in ZFS is memory-consuming. To accommodate this, it is advised to use more than 2 GB of RAM for each terabyte of deduplicated storage.

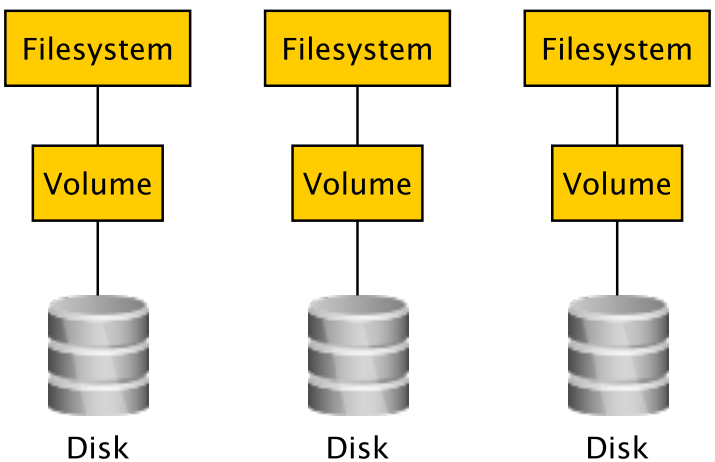
ZFS performance can be further enhanced by utilizing SSDs as write or read cache, creating a so-called hybrid storage pool.

It's essential to note that ZFS is not designed to be a cluster or distributed filesystem. For such use cases, ZFS is not well-suited.

Traditional Storage Architectures vs. ZFS

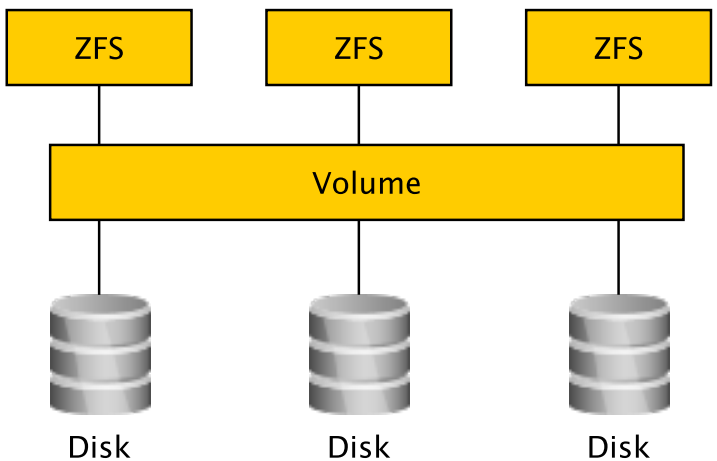
Traditional Architecture

- Partition(s)/Volume per filesystem
- Fragmented capacity and I/O bandwidth
- Manual resizing and growing the storage

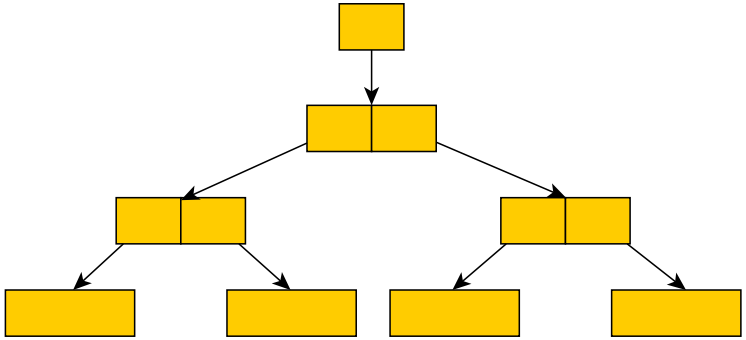


ZFS Architecture

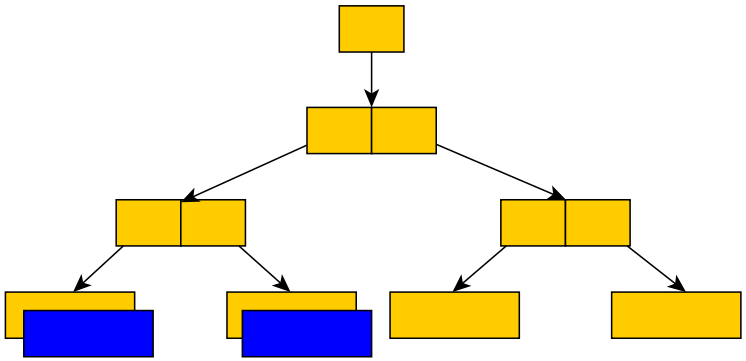
- No partitions required
- Assignment of the total storage to filesystems is flexible
- Integrated error checking
- Snapshots, clones, compression, deduplication



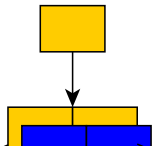
Copy on Write (COW) Transactional Filesystem



1. Consistent state



2. Write some files



Overview

1. Introduction
2. Problems with Today's Filesystems
3. Features of ZFS
   - Simple Administration
   - Quota and Reservations
   - Snapshots
     - Clones
     - Boot Environments
   - Self-Healing Data
   - Compression
   - Deduplication
   - ZFS Volumes
   - ZFS Serialization

Simple Administration

To manage ZFS two commands and their options are necessary: `zpool` and `zfs`.
The `zpool` command manages the underlying storage pool and connected devices. With the `zfs` command one can set all kinds of parameters for the filesystems on top of the pool. Subcommands allow access to all features of ZFS, some of which we will look at in more detail.

Simple Administration – Creating a Storage Pool

To create a storage pool from a single disk use the following command:

```
# zpool create tank /dev/ada1
```

Using the name `tank` (chosen at will) accessing the pool in the future and abstracting away the devices within. Issuing this command causes Further actions to happen automatically:

- Creation of a filesystem with the same name
- Mounting the filesystem under `/tank` (or whatever the name is)
- Storing the settings as part of the pool and reuse them upon reboot
- The storage is instantly available (no lengthy formatting required)

This way, possible errors in `/etc/fstab` do not happen because it remains unchanged. In a pure ZFS system this file is empty. All data used for configuration is part of the pool.

Simple Administration – Displaying Pool Status

Display the status of the storage pool via `zpool status`. A healthy pool looks like this:

```
# zpool status
  pool: tank
 state: ONLINE
  scan: none requested
config:

NAME         STATE     READ WRITE CKSUM
tank         ONLINE       0     0     0
  ada1       ONLINE       0     0     0

errors: No known data errors
```

Display all pools and their capacity using `zpool list`.

```
# zpool list
NAME   SIZE  ALLOC   FREE    CAP  DEDUP  HEALTH  ALTROOT
tank  1016M  89.5K  1016M     0%  1.00x  ONLINE  -
```

Simple Administration – Displaying I/O Statistics

ZFS contains a built-in tool to display I/O statistics (`iostat`) of pools. It shows the free and allocated space as well as the size of read and write operations and their I/O bandwidth.

```
$ zpool iostat
               capacity     operations     bandwidth
pool         alloc   free   read  write   read  write
----------  -----  -----  -----  -----  -----  -----
tank         62.4G  3.56T      0      3  1.19K  15.6K
```

By specifying an interval in seconds as a parameter, statistics display continuously until the user interrupts using Ctrl + C.
An even more detailed display allows the `-v` (verbose) option, displaying I/O for individual pool devices.

Simple Administration – Creating Stripes

A pool with one disk does not provide any redundancy, capacity or even adequate performance. Due to these reasons, use stripes because they offer higher capacity and performance, but **no redundancy**:

```
# zpool create mystripe /dev/ada1 /dev/ada2
```

```
# zpool status
  pool: mystripe
 state: ONLINE
  scan: none requested
config:

        NAME          STATE     READ WRITE CKSUM
        mystripe      ONLINE       0     0     0
          ada1        ONLINE       0     0     0
          ada2        ONLINE       0     0     0

errors: No known data errors
# zpool list
NAME        SIZE  ALLOC   FREE    CAP  DEDUP  HEALTH  ALTROOT
mystripe   1.98G  92.5K  1.98G     0%  1.00x  ONLINE  -
```

Simple Administration – Creating Mirrors (RAID1)

Mirrored disks have less capacity, but ensure redundancy for disk failures and reads can happen in parallel.

```
# zpool create mymirror mirror /dev/ada1 /dev/ada2
```

Using the keyword `mirror`, instructs ZFS to create a mirror from the specified disks.

```
# zpool status
  pool: mymirror
 state: ONLINE
  scan: none requested
config:

        NAME                STATE     READ WRITE CKSUM
        mymirror            ONLINE       0     0     0
          mirror-0          ONLINE       0     0     0
            ada1            ONLINE       0     0     0
            ada2            ONLINE       0     0     0

errors: No known data errors
# zpool list
NAME        SIZE  ALLOC   FREE    CAP  DEDUP  HEALTH  ALTROOT
mymirror   1016M  92.5K  1016M     0%  1.00x  ONLINE  -
```

Simple Administration – Increase Storage Pool Capacity 1/2

To add more devices to the pool and increase capacity without downtime, use `zpool add`. At the moment, our pool looks like this:

```
# zpool status
  pool: mypool
 state: ONLINE
  scan: none requested
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          ada1      ONLINE       0     0     0

errors: No known data errors
# zpool list
NAME     SIZE  ALLOC   FREE    CAP  DEDUP  HEALTH  ALTROOT
mypool   960M    91K   960M     0%  1.00x  ONLINE  -
```

Simple Administration – Increase Storage Pool Capacity 2/2

We add another disk to the pool, striping the data (default setting) over both disks. This increases performance, but is **not** redundant. If any one disk dies, losing *all data* in the pool!

```
# zpool add mypool /dev/ada2
# zpool list mypool
  NAME     SIZE  ALLOC   FREE    CAP  DEDUP  HEALTH  ALTROOT
  mypool  1.90G   145K  1.90G     0%  1.00x  ONLINE  -
# zpool status
  pool: mypool
 state: ONLINE
  scan: none requested
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          ada1      ONLINE       0     0     0
          ada2      ONLINE       0     0     0

errors: No known data errors
```

The newly gained storage space is available without any further configuration.

Simple Administration – Create Mirror from Pool 1/3

Create a mirror from an existing pool with one device using zpool attach. This involves using the `zpool attach` command. To mirror the data on the new device to match the existing device, the pool needs to undergo a *resilvering* process. Resilvering synchronizes both devices to ensure they contain the same data by the end of the operation. Keep in mind that accessing the pool during this process may be slower, but it is still possible.

Simple Administration – Create Mirror from Pool 2/3

```
# zpool attach mypool mirror /dev/da4 /dev/da5
# zpool list
NAME     SIZE  ALLOC   FREE    CAP  DEDUP  HEALTH  ALTROOT
mypool   960M  44.2M   916M     4%  1.00x  ONLINE  -
#
# zpool status mypool
  pool: mypool
 state: ONLINE
status: One or more devices is currently being resilvered.  The pool
        will continue to function, possibly in a degraded state.
action: Wait for the resilver to complete.
  scan: resilver in progress since Mon Nov  5 10:09:42 2012
        5.03M scanned out of 44.1M at 396K/s, 0h1m to go
        5.03M resilvered, 11.39% done
config:

NAME         STATE     READ WRITE CKSUM
mypool       ONLINE       0     0     0
  mirror-0   ONLINE       0     0     0
    da4      ONLINE       0     0     0
    da5      ONLINE       0     0     0  (resilvering)

errors: No known data errors
```

Simple Administration – Create Mirror from Pool 3/3

After the resilver process has finished the status of the pool looks like this:

```
# zpool status
  pool: mypool
 state: ONLINE
  scan: resilvered 44.2M in 0h1m with 0 errors on Mon Nov  5 10:11:34 2012
config:

NAME         STATE     READ WRITE CKSUM
mypool       ONLINE       0     0     0
  mirror-0   ONLINE       0     0     0
    da4      ONLINE       0     0     0
    da5      ONLINE       0     0     0

errors: No known data errors
```

Simple Administration – Create Filesystems/Datasets

After creating the pool with the underlying devices, create filesystems[2] using `zfs create`. Instead of directories, create whole datasets that provide individual options.

```
# zfs create tank/home
```

The newly created dataset is available with the full disk space capacity of the pool, unless specifying other options (more about that later).

```
# zfs list
NAME         USED   AVAIL   REFER   MOUNTPOINT
tank         146K    984M     31K   /tank
tank/home     31K    984M     31K   /tank/home
```

[2] ZFS uses the term dataset, which we're going to use as well from now on. Use of datasets is synonymous with traditional filesystems, although ZFS datasets have more features.

Simple Administration – Display and Edit Properties

Change ZFS properties using `zfs set`.

```
# zfs set atime=off tank
```

Display all available properties with the `zfs get all` command. To see a specific property and the current value provide the name of the property after `zfs get`:

```
# zfs get atime
NAME        PROPERTY   VALUE   SOURCE
tank        atime      off     local
tank/home   atime      off     inherited from tank
```

Similar to object-oriented programming, inheritances in this context are hierarchical. Higher-level objects pass on their properties to those beneath them. This is reflected in the description `inherited from tank` in the `SOURCE` column of the `tank/home` dataset. This approach reduces the need for extensive typing and facilitates setting properties for the entire pool, regardless of the number of datasets. However, exceptions and specific changes can still be made to apply to individual datasets.

Simple Administration – Important Properties

`zfs get all` displays all available properties for a given dataset. Read-only properties have a `-` in their `SOURCE` column, like the creation date (`zfs get creation`) or the used space (`used`).

```
# zfs get creation,used tank
NAME  PROPERTY  VALUE                  SOURCE
tank  creation  Sun May  13 08:13 2012  -
tank  used      634M                   -
```

The user can change other properties. The parent dataset inherits a lot of those at dataset creation time. If not, then default values (shown as `default`) apply.

Simple Administration – Changing Mountpoints

By default, ZFS mounts all new datasets below the name of pool, which forms the root of that pool.

```
# zfs create tank/home
# zfs list
NAME         USED   AVAIL   REFER   MOUNTPOINT
tank         146K    984M     31K   /tank
tank/home     31K    984M     31K   /tank/home
```

When using a different path the following command can change that:

```
# zfs set mountpoint=/usr/home tank/home
# zfs list
NAME         USED   AVAIL   REFER   MOUNTPOINT
tank         146K    984M     31K   /tank
tank/home     31K    984M     31K   /usr/home
```

Note: Specify that the mountpoint with a leading / (as usual in Unix), but the ZFS path in the pool must not have a leading slash.

Simple Administration – helpful properties: exec 1/2

The exec property defines whether files are executable on that dataset. This makes sense in some datasets where executing files would do more harm than good, like in /var/log. Setting this option increases system security for i.e. datasets for the user homes.

```
# zfs create tank/home
# zfs set exec=off tank/home
# zfs create tank/home/susan
# zfs get exec
NAME             PROPERTY  VALUE  SOURCE
tank             exec      on     default
tank/home        exec      off    local
tank/home/susan  exec      off    inherited from tank/home
```

Simple Administration – helpful properties: exec 2/2

From the outside, the ZFS property is not visible and sets the execute (x) flag as usual.

```
$ ls -lah /tank/home
total 5
drwxr-xr-x  3 root  wheel     3B Dec  2 23:01 .
drwxr-xr-x  6 root  wheel     6B Dec  2 22:59 ..
drwx------  2 susan wheel     2B Dec  2 23:00 susan
$ cd susan
susan$ cat myscript.sh
#!/bin/sh
date ; ls
susan$ chmod +x myscript.sh
susan$ ls -l myscript.sh
-rwx--x--x  1 root  wheel  21 Dec  2 23:12 myscript.sh
susan$ ./myscript.sh
unable to execute ./myscript.sh: Permission denied
```

Even root is not allowed to execute files on this dataset, as long as setting the exec property to off.

Simple Administration – helpful properties: `readonly`

Readonly datasets prevent changes (intended or not) by the users. For this use case use `readonly`. Deactivated By default, meaning that datasets mount as read-write.

```
# zfs create -p projects/current
# zfs create projects/finished
# cp -R /home/susan/projects /projects/current
# zfs get readonly projects/finished
NAME                PROPERTY  VALUE   SOURCE
projects/finished   readonly  off     local
# cp /projects/current/photosafari /projects/finished
# zfs set readonly=on projects/finished
# zfs get readonly tank/projects/finished
NAME                PROPERTY  VALUE   SOURCE
projects/finished   readonly  on      local
# cp -R /projects/current/miscphotos /projects/finished
cp: /projects/finished/miscphotos: Read-only file system
```

Backup and archive data are typical candidates for the `readonly` property to protect them from accidental (or intentional) deletion.

Simple Administration – User-defined Properties

Beside the properties set by ZFS, the users can set custom properties and save it as part of the pool configuration.
The syntax is as follows:

```
zfs set name1:name2=value pool
```

The colon distinguishes user-defined variables from the ones that ZFS defines. These custom-properties are helpful when, for example, recording the date of the last backup should or which cost center is paying for the underlying storage disks. Some automatic snapshot solutions rely on these properties to ensure proper functionality. Individual datasets inherit and change these as defined by the user.

```
# zfs set hda:firstlecture=20.04.2013
# zfs get hda:firstlecture
NAME  PROPERTY           VALUE              SOURCE
tank  hda:firstlecture   20.04.2013         local
```

Destroying Datasets

Remove datasets, pools, snapshots, and clones using `zfs destroy` or `zpool destroy` losing all data on these objects. When child-datasets exist ZFS warns the user and offers an option to recursively walk through the child objects.

```
# zfs list
NAME         USED   AVAIL   REFER   MOUNTPOINT
tank         146K    984M     31K   /tank
tank/home     31K    984M     31K   /tank/home
# zfs destroy tank
cannot destroy 'tank': filesystem has children
use '-r' to destroy the following datasets:
tank/home
```

Overview

1. Introduction
2. Problems with Today's Filesystems
3. Features of ZFS
   - Simple Administration
   - Quota and Reservations
   - Snapshots
     - Clones
     - Boot Environments
   - Self-Healing Data
   - Compression
   - Deduplication
   - ZFS Volumes
   - ZFS Serialization

Quota and Reservations

As we've discussed previously, the entire capacity of the pool is accessible to all ZFS datasets. Unlike traditional filesystems that rely on partitions to limit disk space, ZFS employs **quotas**to dynamically restrict disk space usage per dataset. This allows, for instance, setting size limits for home directories (more like home datasets) to prevent a user from using up all available disk space, ensuring fairness for other users on the system.
In certain scenarios, it might be necessary to **reserve** a specific amount of disk space for a user or dataset. ZFS ensures that the designated space is always available, regardless of how much other users have already utilized. It's important to note that reservations cannot exceed the total amount of disk space available in the system.

Using ZFS Quotas to Limit Disk Space 1/3

We want to set a quota for the created home datasets. By default, every user can make use of all the disk space that the pool provides, there is no quota applied. At the moment, disk space usage is as follows:

```
$ df -h
Filesystem             Size    Used   Avail Capacity  Mounted on
hda                    1.3G     31k    1.3G     0%    /hda
hda/home               1.3G     97k    1.3G     0%    /usr/home
hda/home/bcr           1.3G     84k    1.3G     0%    /usr/home/bcr
$ zfs get quota
NAME                   PROPERTY  VALUE  SOURCE
hda                    quota     none   default
hda/home               quota     none   default
hda/home/bcr           quota     none   default
```

Using ZFS Quotas to Limit Disk Space 2/3

We now set a quota of 500 MB on `hda/home/bcr` using the `zfs set quota` command.

```
# zfs set quota=500m hda/home/bcr
```

```
$ df -h
Filesystem            Size    Used   Avail Capacity  Mounted on
hda                   1.3G     31k     1.3G     0%    /hda
hda/home              1.3G     97k     1.3G     0%    /usr/home
hda/home/bcr          500M     84k     499M     0%    /usr/home/bcr
$ zfs get quota
NAME                  PROPERTY  VALUE  SOURCE
hda                   quota     none   default
hda/home              quota     none   default
hda/home/bcr          quota     500M   local
```

The quota does apply to the specified dataset. By using inheritance, set the global quota for all home directories. This way, datasets that created in the future will automatically inherit these size limits.

Using ZFS Quotas to Limit Disk Space 3/3

We now write a big file (one that is bigger than the available disk space) in `hda/home/bcr` to test the quota enforcement.

```
# dd if=/dev/urandom of=./bigfile bs=1g
...
dd: ./bigfile: Disc quota exceeded
1+0 records in
0+1 records out
512020480 bytes transferred in 94.152408 secs (5438209 bytes/sec)
% ls -lah bigfile
-rw-r--r--  1 bcr  bcr   499M Nov 30 15:46 bigfile
$ df -h
Filesystem         Size     Used    Avail Capacity  Mounted on
hda                1.3G      31k     1.3G     0%    /hda
hda/home           1.3G      97k     1.3G     0%    /usr/home
hda/home/bcr       500M     500M       0B   100%    /usr/home/bcr
```

The quota rules are strictly enforced by ZFS. There can be no more space allocated than specified in the quota. The user either has to delete some old files, compress them or ask for a quota increase.
Deactivate a quota using `zfs set quota=none` on the dataset.

ZFS Reservations for Disk Space Guarantees 1/3

Often, unpredictable how much space a specific partition is going to use. This is bad for non-ZFS filesystems when the allocated space is not enough and requires reformatting and reinstallation because the initial assumptions were wrong.
Reservations set in ZFS give a specific amount of disk space to a dataset no matter how much other datasets require. The syntax for reservations is:

```
zfs set reservation=size
```

ZFS Reservations for Disk Space Guarantees 2/3

We want to ensure that the home dataset has a guaranteed 500 MB of disk space available.
This is what we have:

```
$ df -h
Filesystem             Size    Used   Avail Capacity  Mounted on
hda                    1.3G     31k     1.3G     0%    /hda
hda/home               1.3G     97k     1.3G     0%    /usr/home
hda/home/bcr           1.3G     84k     1.3G     0%    /usr/home/bcr
# zfs set reservation=500m hda/home/bcr
$ df -h
Filesystem             Size    Used   Avail Capacity  Mounted on
hda                    801M     31k     801M     0%    /hda
hda/home               801M     97k     801M     0%    /usr/home
hda/home/bcr           1.3G     82k     1.3G     0%    /usr/home/bcr
$ zfs get reservation
NAME                   PROPERTY     VALUE    SOURCE
hda                    reservation  none     default
hda/home               reservation  none     default
hda/home/bcr           reservation  500M     local
```

The output of `df` is not too accurate and can be confusing at first.
Reducing the total pool capacity by 500 MB, but the dataset with the reservation still reports the original 1.3 GB space available.

ZFS Reservations for Disk Space Guarantees 3/3

```
$ df -h
Filesystem          Size    Used   Avail Capacity  Mounted on
hda                 801M     31k    801M     0%    /hda
hda/home            801M     97k    801M     0%    /usr/home
hda/home/bcr        1.3G     82k    1.3G     0%    /usr/home/bcr
```

When other datasets use more disk space the size of `hda/home/bcr` decreases as well. 500 MB will still be available to that dataset.

```
# zfs create hda/home/myboss
myboss$ dd if=/dev/urandom of=./hda/home/myboss/bigfile bs=700m
dd: ./bigfile: No space left on device
2+0 records in
1+1 records out
781864960 bytes transferred in 176.822419 secs (4421752 bytes/sec)
$ df -h
Filesystem          Size    Used   Avail Capacity  Mounted on
hda                   0M     31k      0M   100%    /hda
hda/home              0M     97k      0M   100%    /usr/home
hda/home/bcr        500M     82k    500M     0%    /usr/home/bcr
hda/home/myboss       0M    700M      0M   100%    /usr/home/myboss
```

Reservations ensure disk space is available even in situations where pool space is short.

Combining ZFS Quotas and Reservations

Combine ZFS quotas and reservations to ensure availability of disk space while at the same time ZFS takes care of using no more than the amount of reserved disk space.

```
# LIMIT=500m
# zfs set quota=$LIMIT hda/home/bcr
# zfs set reservation=$LIMIT hda/home/bcr
$ df -h
Filesystem          Size    Used   Avail Capacity  Mounted on
hda                 791M     31k    791M     0%    /hda
hda/home            791M     97k    791M     0%    /usr/home
hda/home/bcr        500M     82k    500M     0%    /usr/home/bcr
```

This output corresponds to the output of traditional filesystem partitions with regard to space usage.

Overview

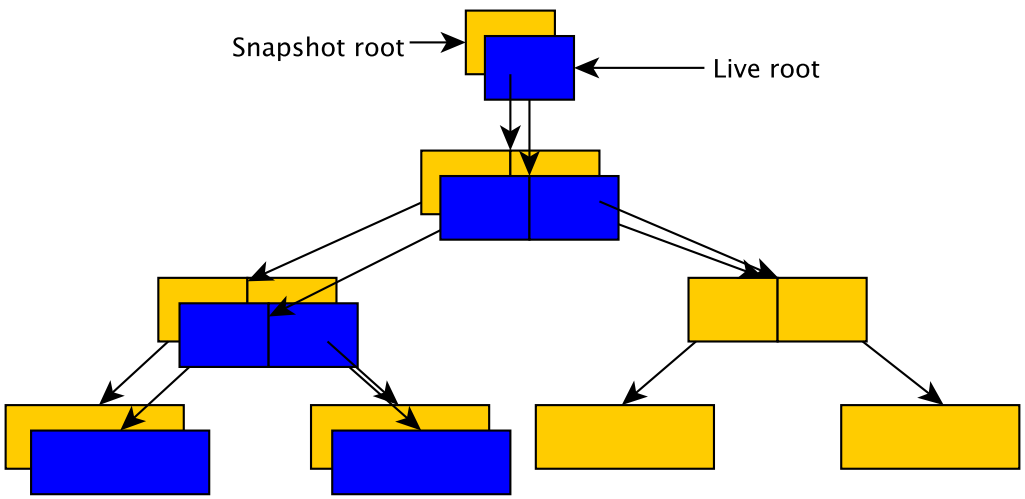
1. Introduction
2. Problems with Today's Filesystems
3. Features of ZFS
   - Simple Administration
   - Quota and Reservations
   - Snapshots
     - Clones
     - Boot Environments
   - Self-Healing Data
   - Compression
   - Deduplication
   - ZFS Volumes
   - ZFS Serialization

Snapshots

A **snapshot** is a read-only copy of an entire dataset or volume, created and utilized for various scenarios. It's important to note that snapshots are created for whole datasets, not individual files or directories. When a snapshot is taken of a dataset, it includes all files and directories within that dataset at the specific point in time when the snapshot was created. Snapshots serve as a convenient backup before potentially risky actions. For instance, *before* undertaking actions like system upgrades, software installations, or tests involving the `rm` command, taking a snapshot is a prudent step. If an error or mistake occurs, you can easily roll back to the snapshot. This returns the dataset to the state it was in at the time of snapshot creation, without requiring a system reboot, as if the risky action never took place.

ZFS Snapshots

Implicit snapshots, created through the underlying copy-on-write (COW) model, occur automatically each time parts of the filesystem tree are copied (or more precisely, pointers are preserved). In the absence of an explicit snapshot, the COW model discards the copied data. When a snapshot is created, ZFS retains the old version of the filesystem state for future use. Importantly, creating snapshots in ZFS *does not incur additional costs* because they are essentially a by-product of normal filesystem operations and are instantly available.



Creating Snapshots

To create snapshots in ZFS, use the following syntax:

```
zfs snapshot dataset@name
```

`name` can be any identifier (i.e. date and time). The snapshot creation happens at once. Display all snapshots using the parameter `-t snapshot` with `zfs list`.

```
# zfs snapshot hda/home/bcr@backup
# zfs list -t snapshot
NAME                    USED  AVAIL  REFER  MOUNTPOINT
hda/home/bcr@backup        0      -  85.5K  -
```

The display shows that the snapshot is not mounted on the filesystem, which is why there is no path specified below `MOUNTPOINT`. Also no mention of available disk space (`AVAIL` column), since snapshots are read-only.

Working with Snapshots

When comparing a snapshot with its dataset, the space usage becomes clear.

```
# zfs list -rt all hda/home/bcr
NAME                   USED  AVAIL  REFER  MOUNTPOINT
hda/home/bcr          85.5K  1.29G  85.5K  /usr/home/bcr
hda/home/bcr@backup       0      -  85.5K  -
```

Another feature of ZFS snapshots shows here. Snapshots store the changes (delta) after creation time and the previous (if any) snapshot and not the complete contents of the filesystem all over again to save space. This means that another snapshot of an unchanged dataset does not require any extra space!

```
# cp /etc/passwd /usr/home/bcr
# zfs snapshot hda/home/bcr@after_cp
# zfs list -rt all hda/home/bcr
NAME                    USED  AVAIL  REFER  MOUNTPOINT
hda/home/bcr            115K  1.29G    88K  /usr/home/bcr
hda/home/bcr@backup      27K      -  85.5K  -
hda/home/bcr@after_cp      0      -    88K  -
```

Working with Snapshots - Differences Between Snapshots

When the user wants to know the differences between two snapshots, then ZFS can display them using the `zfs diff` command. For our example it would look like this:

```
# zfs diff hda/home/bcr@backup
M /usr/home/bcr/
M /usr/home/bcr/.histfile
+ /usr/home/bcr/passwd
```

The following table describes the status column:

Character	Description
+	File added
−	File deleted
M	File changed
R	File renamed

Working with Snapshots - Rollback

When it becomes necessary to roll back to the state that a snapshot holds, using a command known from database systems: `zfs rollback`. The syntax is:

```
zfs rollback snapshot
```

Rolling back a snapshot in ZFS involves discarding all changes made to files since the snapshot's creation, effectively reverting the filesystem to its state at the time of the snapshot.
By default, ZFS will roll back to the most recent snapshot. If you want to roll back to an older snapshot than the current one, you need to destroy the snapshots in between. To do this, use the `-r` (recursive) option; otherwise, ZFS will issue a warning.
To preserve a snapshot that lies between the current state of the filesystem and the one you want to roll back to, consider creating a clone. Clones offer a way to maintain a specific snapshot's state alongside the current filesystem state. For more information on clones, refer to the section dedicated to this topic.

Working with Snapshots - Rollback Example

In this example we roll back to the last snapshot because of an accidental delete of an important file.

```
# rm /hda/home/bcr/*
# ls
# zfs rollback hda/home/bcr@after_cp
# ls
passwd loveletter.txt
# zfs list -t snapshot
NAME                      USED  AVAIL  REFER  MOUNTPOINT
hda/home/bcr@backup        27K      -  85.5K  -
hda/home/bcr@after_cp        0      -    88K  -
```

The rollback operation succeeded in restoring all files from the snapshot!

Working with Snapshots - Rollback To First Snapshot

Now we want to roll back to the first snapshot, because we don't want to have the file `passwd` around anymore[3].

```
# ls
passwd loveletter.txt
# zfs list -t snapshot
NAME                       USED  AVAIL  REFER  MOUNTPOINT
hda/home/bcr@backup         27K      -  85.5K  -
hda/home/bcr@after_cp         0      -    88K  -
# rm /hda/home/bcr/*
# ls
# zfs rollback hda/home/bcr@backup
cannot rollback to 'hda/home/bcr@backup': more recent snapshots exist
use '-r' to force deletion of the following snapshots:
hda/home/bcr@after_cp
# zfs rollback -r hda/home/bcr@backup
# ls
loveletter.txt
# zfs list -t snapshot
NAME                     USED  AVAIL  REFER  MOUNTPOINT
hda/home/bcr@backup       27K      -  85.5K  -
```

[3] and we are too lazy to delete it. ;-)

Working with Snapshots - Restoring a File

What about situations where not the whole snapshot should roll back, but restoring a couple of files from that state? In this case, ZFS keeps a special hidden (even a `ls -a` won't display it) directory `.zfs` on every dataset. This behavior can change by setting the `snapdir` property to `visible`: `zfs set snapdir=visible pool`.
We assume for this example that the snapshot from the previous slide was not rolled back.

```
# ls .zfs/snapshot
after_cp backup
# ls .zfs/snapshot/after_cp
passwd
# cp .zfs/snapshot/after_cp/passwd /hda/home/bcr
```

Copy single files or directories from this hidden directory. Copying files *into* that directory will fail due to the read-only nature of snapshots.

```
# cp /hda/home/bcr/loveletter.txt .zfs/snapshot/after_cp/passwd
cp: .zfs/snapshot/after_cp/loveletter.txt: Read-only file system
```

Overview

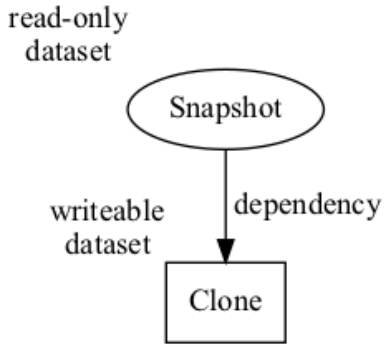
1. Introduction
2. Problems with Today's Filesystems
3. Features of ZFS
   - Simple Administration
   - Quota and Reservations
   - Snapshots
     - Clones
     - Boot Environments
   - Self-Healing Data
   - Compression
   - Deduplication
   - ZFS Volumes
   - ZFS Serialization

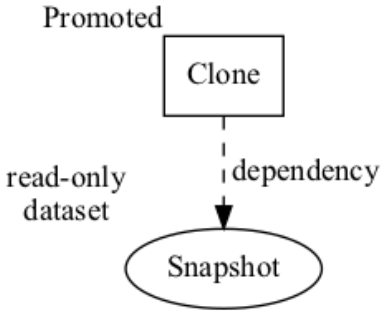
Snapshot Clones

Clones represent a *writeable* copy of a snapshot. Treat them as a self-contained dataset. To create a clone from a snapshot use the following syntax:

```
zfs clone snapshotname newdataset
```

When creating a clone, ZFS creates an implicit dependency between clone and the snapshot. Due to that, destroying the snapshot is impossible after creating a clone that depends on it.





To remove this dependency, "promote" a clone to a real dataset using `zfs promote nameoftheclone`. The snapshot and clone switch their dependency around and removing the snapshot is possible now. Mount the clone at any position within the ZFS filesystem except at the original path of the snapshot.

Working with Snapshots - Creating Clones from Snapshots

We assume the following filesystem is present:

```
# zfs list -rt all geonosis/home/boba
NAME                              USED  AVAIL  REFER  MOUNTPOINT
geonosis/home/boba                108K  1.29G    87K  /geonosis/home/boba
geonosis/home/boba@backup          21K      -  85.5K  -
geonosis/home/boba@after_cp         0      -    87K  -
```

Create a clone now based on the state of the filesystem after the last snapshot.

```
# zfs clone geonosis/home/boba@after_cp geonosis/home/trooper
# ls /geonosis/home/*
geonosis/home/boba
darthsidious.txt
geonosis/home/trooper
darthsidious.txt
# df -h
Filesystem                Size     Used    Avail Capacity  Mounted o
geonosis                  1.3G      31k     1.3G      0%    /geonosis
geonosis/home             1.3G      98k     1.3G      0%    /geonosis
geonosis/home/boba        1.3G      87k     1.3G      0%    /geonosis
geonosis/home/trooper     1.3G      87k     1.3G      0%    /geonosis
```

Working with Snapshots - Promoting Clones

By cloning the created dataset it inherits all properties that the snapshot had. For demonstration purposes, we copy a file into the writeable clone.

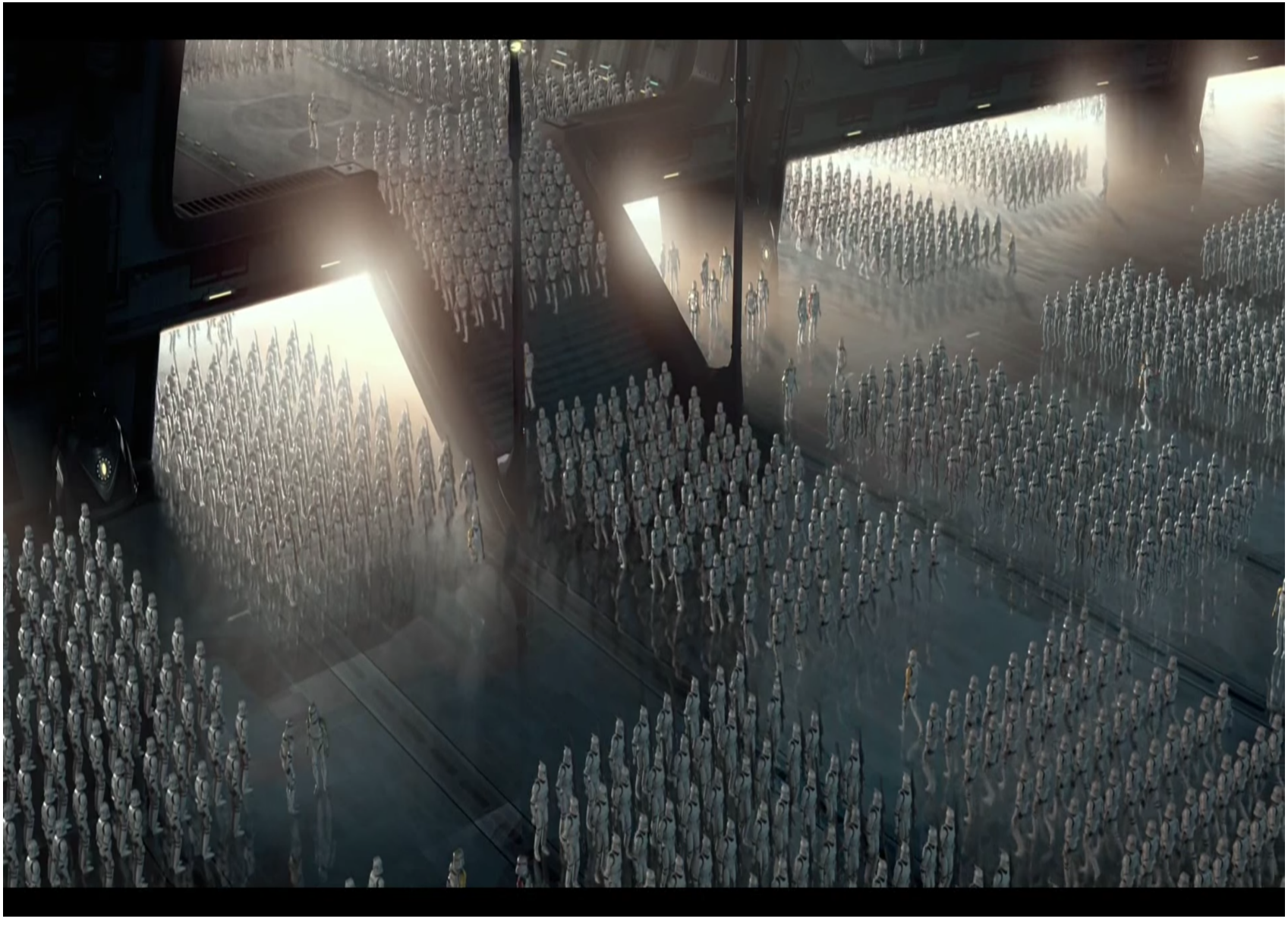
```
# cp neworders.txt /geonosis/home/trooper
# zfs get origin geonosis/home/trooper
NAME                      PROPERTY  VALUE                          SOURCE
geonosis/home/trooper     origin    geonosis/home/boba@after_cp    -
# zfs promote geonosis/home/trooper
# zfs get origin geonosis/home/trooper
NAME                      PROPERTY  VALUE    SOURCE
geonosis/home/trooper     origin    -        -
```

The clone should now serve as the new home directory. The clone must not have the same name as any existing snapshot. Change the name of a dataset using `zfs rename`.

```
# zfs destroy -f geonosis/home/boba
# zfs rename geonosis/home/trooper geonosis/home/stormtrooper
# ls /geonosis/home/stormtrooper
darthsidious.txt neworders.txt
```

We got the file `neworders.txt` from the `geonosis/home/trooper` clone.

Magnificent, aren't they?



Overview

1. Introduction
2. Problems with Today's Filesystems
3. Features of ZFS
   - Simple Administration
   - Quota and Reservations
   - Snapshots
     - Clones
     - Boot Environments
   - Self-Healing Data
   - Compression
   - Deduplication
   - ZFS Volumes
   - ZFS Serialization

Boot Environments

Boot Environments (BE) extend the concept of clones, where writeable copies of a snapshot are applied to the entire operating system. In a ZFS-only installation, the operating system can maintain different versions at a specific point in time, allowing for seamless switching between them. This switch can occur during boot time or by explicitly selecting a Boot Environment on the command line for the next boot. BEs provide a valuable mechanism for handling issues such as failed system upgrades, problematic package installations, or security threats like ransomware. By rolling back to a previous BE (provided one exists), significant time is saved, eliminating the need for reinstalling and restoring the system from backup in the event of a disaster.
Temporarily mounting a Boot Environment is a useful practice for testing updates. If the update is successful, the BE can become the new system. If not, changes can be safely discarded without affecting the running system copy.
Boot environments also facilitate switching between different work environments, such as kernel development or regular usage. Since clones, on which boot environments are built, initially occupy minimal disk space, keeping multiple BEs around is feasible. Their size only grows

Creating and Listing Boot Environments

Boot environments reside within the *poolname*/ROOT dataset path. A standard boot environment exists on FreeBSD after the installation. Create other BEs using `bectl create` *nameofBE*.
Example:

```
$ zfs list
  ...
sys/ROOT/default
sys/ROOT/pre-upgrade
  ...
$ bectl list
BE             Active Mountpoint Space Created
default        NR     /          2.54G 2021-07-13 11:48
pre-upgrade    -      -          16K   2021-11-29 09:21
```

The `Active` column in the `bectl list` output contains `N` when this boot environment is active at the moment (now). The `R` means that it will become active upon the next reboot.

Changing Active Boot Environment

There can ever be only one boot environment active. Switch BEs using the boot loader by selecting option 8, then toggle through available BEs in the sub-menu. After the boot, the `bectl list` output changed to this:

```
$ bectl list
BE             Active Mountpoint Space Created
default        R      /          2.54G 2021-07-13 11:48
pre-upgrade    N      -          16K   2021-11-29 09:21
```

Since the `R` is still on the BE named `default`, the system will continue booting from that on the next boot. To permanently change BEs to another, use `bectl activate` *`nameofbe`*.
This will move both the `R` flag from `default` to the activated BE, as well as the /-mountpoint and uses this BE's disk space:

```
$ bectl list
BE             Active Mountpoint Space Created
default        -      -          16K   2021-07-13 11:48
pre-upgrade    NR     /          2.54G 2021-11-29 09:21
```

Boot Environment Layout

The ZFSBE ZFS property shows which OS directories are part of the BE:

```
1  $ zfs get -r canmount zroot
2  NAME                  PROPERTY  VALUE     SOURCE
3  zroot                 canmount  on        default
4  zroot/ROOT            canmount  on        default
5  zroot/ROOT/default    canmount  noauto    local
6  zroot/tmp             canmount  on        default
7  zroot/usr             canmount  off       local
8  zroot/usr/home        canmount  on        default
9  zroot/usr/ports       canmount  on        default
10 zroot/usr/src         canmount  on        default
11 zroot/var             canmount  off       local
12 zroot/var/audit       canmount  on        default
13 zroot/var/crash       canmount  on        default
14 zroot/var/log         canmount  on        default
15 zroot/var/mail        canmount  on        default
16 zroot/var/tmp         canmount  on        default
```

The filesystems /usr (line 7) and /var (line 11) have the canmount property set to off. This way, they are not a separate dataset, but part of / for each boot environment created. This is also the reason why data in /usr/home (user's home directories) survive switching between boot environments, as they have their own dataset. [ZFSBE]

Practical Example: pkg upgrade in Boot Environment 1/2

We want to perform a package upgrade to see if it breaks our currently installed version. We do this in a separate boot environment called pkg-upgrade. First we create it:

```
# bectl create pkg-upgrade
```

Next, we mount this boot environment into the system under its name:

```
# bectl mount pkg-upgrade
Successfully mounted pkg-upgrade at /tmp/be_mount.9IqH
```

Note the path /tmp/be_mount.9IqH that was provided as the location of the mountpoint. The mount command confirms the mounted boot environment:

```
$ mount
...
sys/ROOT/pkg-upgrade on /tmp/be_mount.9IqH (zfs, local, noatime, nfsv4acls
```

Practical Example: pkg upgrade in Boot Environment 2/2

We can now point pkg to use the boot environment using the -r parameter.

```
# pkg -r /tmp/be_mount.9IqH upgrade
Updating FreeBSD repository catalogue...
Fetching meta.conf: 100%    163 B   0.2kB/s    00:01
Fetching packagesite.pkg: 100%    6 MiB   6.7MB/s    00:01
Processing entries: 100%
FreeBSD repository update completed. 31367 packages processed.
All repositories are up to date.
Checking for upgrades (2 candidates): 100%
Processing candidates (2 candidates): 100%
The following 2 package(s) will be affected (of 0 checked):

Installed packages to be UPGRADED:
 ...
```

Once the update finishes, boot into the pkg-upgrade boot environment and test it. When satisfied and everything works, activate this BE permanently. Keep old BEs around to go back to them if defects are found later. [KLARABEMGMT]

Practical Example: Testing an Operating System Upgrade 1/3

We can also safely perform a system upgrade inside a boot environment without affecting the current operating system. Create a fresh boot environment for the update, named after the new version:

```
# bectl create 13.2
# bectl mount 13.2
Successfully mounted 13.2 at /tmp/be_mount.e3lu
```

We can now use `freebsd-update` as if we would update the current operating system. This takes three parameters:

- `-b` (basedir) points to the boot environment
- `-d` (working directory) is used by `freebsd-update` to store work files
- `-r` specifies the release to upgrade to

```
# freebsd-update -b /tmp/be_mount.e3lu -d /tmp/be_mount.e3lu/var/db/freebs
 upgrade -r 13.2-RELEASE
```

Practical Example: Testing an Operating System Upgrade 2/3

The upgrade starts comparing the OS files in the boot environment with the online reference system:

```
The following file will be removed, as it no longer exists in
FreeBSD 13.2-RELEASE: ...
Does this look reasonable (y/n)? y
...
To install the downloaded upgrades, run "/usr/sbin/freebsd-update install"
```

Use the `install` keyword to `freebsd-update` as instructed, again pointing to the boot environment using the same parameters.

```
# freebsd-update -b /tmp/be_mount.e3lu -d /tmp/be_mount.e3lu/var/db/freebsd
 install -r 13.2-RELEASE
Installing updates...
Kernel updates have been installed. Please reboot and run
"/usr/sbin/freebsd-update install" again to finish installing updates.
```

Practical Example: Testing an Operating System Upgrade 3/3

We could now boot into the new boot environment and finish the update in there as instructed by `freebsd-update`. But what if the BE does not boot properly, even at this stage of the upgrade? The `bectl activate` command has a `-t` flag to only temporarily activate this boot environment *once*. On the next reboot, the previous BE will be active again. Perfect for testing an update and getting back to a working system if things break:

```
# bectl activate -t 13.2
# reboot
```

Reboot and the 13.2 boot environment automatically boots and clears the temporary flag. Since we're in the new system BE, we can finish the update there:

```
# freebsd-update install
```

Once done and the upgrade seems to be working, activate it permanently (without the `-t` flag). If the upgrade did not work, the old BE will become active upon the next reboot again. This is a safe and easy way

Overview

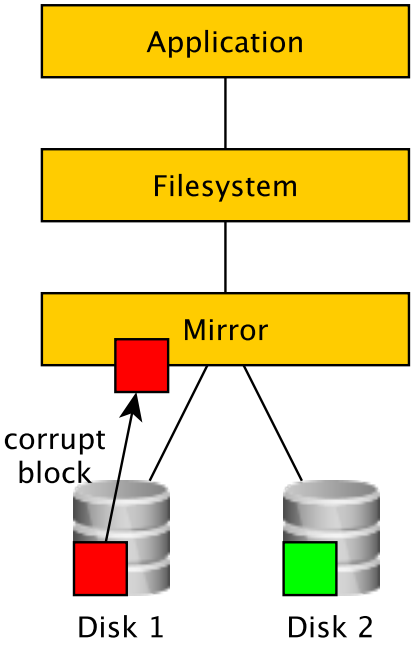
1. Introduction
2. Problems with Today's Filesystems
3. Features of ZFS
   - Simple Administration
   - Quota and Reservations
   - Snapshots
     - Clones
     - Boot Environments
   - Self-Healing Data
   - Compression
   - Deduplication
   - ZFS Volumes
   - ZFS Serialization

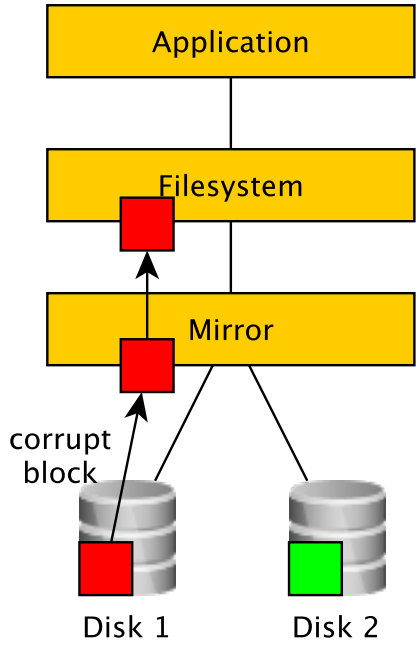
Self-Healing Data

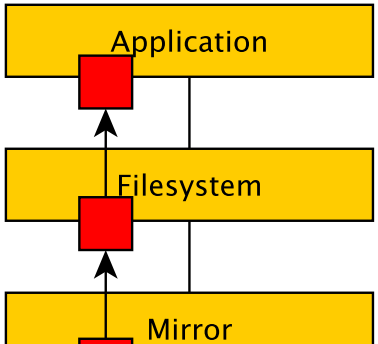
As mentioned earlier, traditional filesystems may not execute their operations atomically. Consequently, after a system crash, resolving inconsistent metadata is typically done using `fsck` before remounting. This process results in prolonged offline periods for storage and inevitable issues such as the loss of non-persistent data in RAM (not yet written) and inconsistent mirror devices (commonly referred to as split-brain). ZFS, on the other hand, has the capability to detect and correct corrupted data based on checksums during regular operation. Therefore, there is no need for `fsck`. Checks for consistency, carried out through `zpool scrub`, can be performed during periods of low storage demand, such as at night. This eliminates the necessity for scheduled downtimes, ensuring that data remains accessible at all times. While Read/Write operations may slow down during the scrub operation, they remain possible, providing continuous availability of data without the need for scheduled downtime.

Traditional Filesystems Do Not Detect All Errors

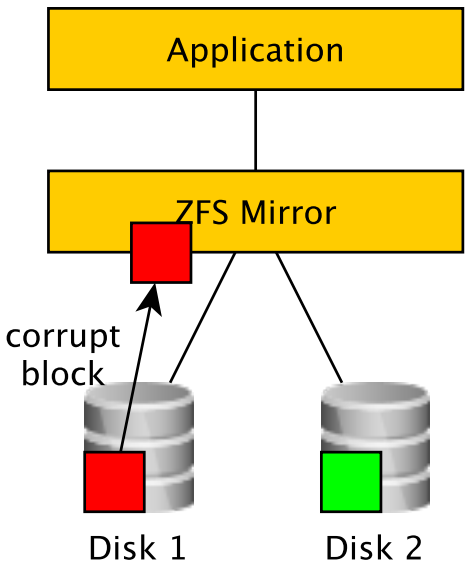
In traditional redundantly held data (i.e. RAID1), the problem of silent data corruption like bit-flips is evident.



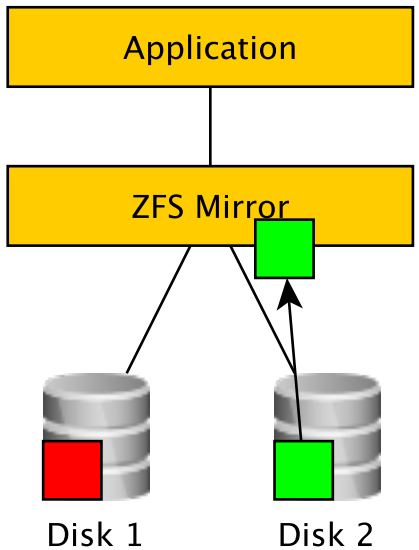




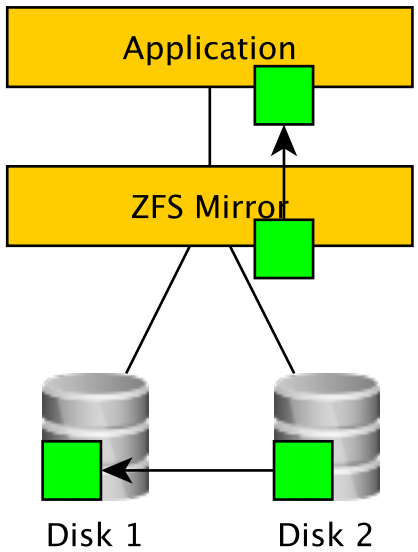
Self-Healing Data in ZFS via Redundancy



1. Reading corrupt block



2. ZFS detects wrong checksum, reads mirror with correct checksum



3. Supply correct data of the mirror device to the application **and** correct the corrupt block!

ZFS Self-Healing Demo

This demonstrates the example of the previous slide on a real ZFS pool. To do that, we create a mirror and copy some data to it. Computing a checksum as a reference for the correct pool data. Export the pool afterwards and write some random data on one device of the pool to simulate the data corruption. After that, import the pool again and display the pool status. Create a second checksum to compare it with the first one. When they match, then ZFS properly detected and corrected the data automatically using the other device in the mirror.
Note that this works when there is enough redundancy in the pool already. Either by using a redundant setup with enough devices (RAID-1, RAIDZ) or by setting the `copies` property to 2 or higher *before* writing data to the pool.

ZFS Self-Healing Demo 1/6

```
# zpool create healer mirror da0 da1
# zpool status
  pool: healer
 state: ONLINE
  scan: none requested
config:

 NAME         STATE     READ WRITE CKSUM
 healer       ONLINE       0     0     0
   mirror-0   ONLINE       0     0     0
     da0      ONLINE       0     0     0
     da1      ONLINE       0     0     0
 errors: No known data errors
# zpool list
NAME     SIZE  ALLOC   FREE    CAP  DEDUP  HEALTH  ALTROOT
healer   960M  92.5K   960M     0%  1.00x  ONLINE  -
# cp /some/important/data /healer
# zfs list
NAME     SIZE  ALLOC   FREE    CAP  DEDUP  HEALTH  ALTROOT
healer   960M  67.7M   892M     7%  1.00x  ONLINE  -
# sha1 /healer > checksum.txt
# cat checksum.txt
SHA1 (/healer) = 2753eff56d77d9a536ece6694bf0a82740344d1f
```

ZFS Self-Healing Demo 2/6

Warning:

This dd example can destroy data when used on the wrong device or a non-ZFS filesystem.

Use it at your own risk and make proper backups first!

```
# zpool export healer
# dd if=/dev/random of=/dev/da1 bs=1m count=200
200+0 records in
200+0 records out
209715200 bytes transferred in 62.992162 secs (3329227 bytes/sec)
# zpool import healer
```

ZFS Self-Healing Demo 3/6

```
# zpool status
  pool: healer
 state: ONLINE
status: One or more devices has experienced an unrecoverable error.  An
        attempt was made to correct the error.  Applications are unaffected.
action: Determine if the device needs to be replaced, and clear the errors
        using 'zpool clear' or replace the device with 'zpool replace'.
   see: http://www.sun.com/msg/ZFS-8000-9P
  scan: none requested
config:

 NAME        STATE     READ WRITE CKSUM
 healer      ONLINE       0     0     0
   mirror-0  ONLINE       0     0     0
     da0     ONLINE       0     0     0
     da1     ONLINE       0     0     1

 errors: No known data errors
```

We compare the checksums of the pool before and after the import.

```
# sha1 /healer >> checksum.txt
# cat checksum.txt
SHA1 (/healer) = 2753eff56d77d9a536ece6694bf0a82740344d1f
SHA1 (/healer) = 2753eff56d77d9a536ece6694bf0a82740344d1f
```

ZFS Self-Healing Demo 4/6

```
# zpool scrub healer
# zpool status
  pool: healer
 state: ONLINE
status: One or more devices has experienced an unrecoverable erro
An
        attempt was made to correct the error.  Applications are
action: Determine if the device needs to be replaced, and clear t
        using 'zpool clear' or replace the device with 'zpool rep
   see: http://www.sun.com/msg/ZFS-8000-9P
  scan: scrub in progress since Mon Dec 10 12:23:30 2012
        10.4M scanned out of 67.0M at 267K/s, 0h3m to go
        9.63M repaired, 15.56% done
config:

 NAME          STATE     READ WRITE CKSUM
 healer        ONLINE       0     0     0
   mirror-0    ONLINE       0     0     0
     da0       ONLINE       0     0     0
     da1       ONLINE       0     0   627  (repairing)

errors: No known data errors
...
```

ZFS Self-Healing Demo 5/6

```
# zpool status
  pool: healer
 state: ONLINE
status: One or more devices has experienced an unrecoverable erro
An
        attempt was made to correct the error.  Applications are
action: Determine if the device needs to be replaced, and clear t
        using 'zpool clear' or replace the device with 'zpool rep
   see: http://www.sun.com/msg/ZFS-8000-9P
  scan: scrub repaired 66.5M in 0h2m with 0 errors on Mon Dec 10
config:

 NAME          STATE     READ WRITE CKSUM
 healer        ONLINE       0     0     0
   mirror-0    ONLINE       0     0     0
     da0       ONLINE       0     0     0
     da1       ONLINE       0     0 2.72K

 errors: No known data errors
```

ZFS Self-Healing Demo 6/6

```
# zpool clear healer
# zpool status
  pool: healer
 state: ONLINE
  scan: scrub repaired 66.5M in 0h2m with 0 errors on Mon Dec 10
config:

 NAME        STATE     READ WRITE CKSUM
 healer      ONLINE       0     0     0
   mirror-0  ONLINE       0     0     0
     da0     ONLINE       0     0     0
     da1     ONLINE       0     0     0

 errors: No known data errors
```

Overview

1. Introduction
2. Problems with Today's Filesystems
3. Features of ZFS
   - Simple Administration
   - Quota and Reservations
   - Snapshots
     - Clones
     - Boot Environments
   - Self-Healing Data
   - Compression
   - Deduplication
   - ZFS Volumes
   - ZFS Serialization

Compression

A classic method for saving disk space involves compressing files using tools like `zip` and others. ZFS offers the ability to compress individual datasets, automatically compressing data when saved and decompressing it when loaded. There's no need to use an additional tool for this process, and it allows the filesystem to store more data efficiently.
When compression is activated for a ZFS dataset, all **new data** stored will be compressed. Pre-existing data in that dataset will maintain their original size, except when modified or stored again while compression is active.
Different compression algorithms are available, producing varied results based on the data type being compressed. While compression may impose a slightly higher CPU load, this impact should be manageable in today's multi-processor systems.

ZFS Compression - Usage Example 1/2

The FreeBSD Ports Tree contains a lot of text files (Makefiles, patches, description files, etc.), which compresses well. First, we create the necessary directory structure:

```
# zfs create tank/usr
# zfs create tank/usr/ports
# zfs create -o mountpoint=/usr/ports tank/usr/ports
```

Then we set the compression to the GZIP algorithm; this could have happened when creating the datasets above using the -o option.

```
# zfs set compression=gzip tank/usr/ports
# zfs get compressratio tank/usr/ports
NAME             PROPERTY       VALUE  SOURCE
tank/usr/ports   compressratio  1.00x  -
```

Now we download a current copy of the ports tree into /usr/ports.

```
# portsnap auto
```

After this operation finishes check the compression ratio again.

```
# zfs get compressratio tank/usr/ports
NAME             PROPERTY       VALUE  SOURCE
tank/usr/ports   compressratio  3.02x  -
```

ZFS Compression - Usage Example 2/2

To check the space savings, we create another uncompressed dataset for comparison:

```
# zfs create -o compression=off tank/usr/uncompressed
# zfs get compressratio,compression tank/usr/uncompressed
NAME                      PROPERTY       VALUE  SOURCE
tank/usr/uncompressed     compressratio  1.00x  -
tank/usr/uncompressed     compression    off    local
```

Then we copy the source tree from /usr/ports to the new dataset.

```
# cp -R /usr/ports/* /usr/uncompressed
# zfs get compressratio,compression tank/usr/uncompressed
NAME                      PROPERTY       VALUE  SOURCE
tank/usr/uncompressed     compressratio  1.00x  -
tank/usr/uncompressed     compression    off    local
```

We can compare space usage now (unimportant fields stripped from output).

```
# zfs list -o space tank/usr/uncompressed tank/usr/ports
NAME                      AVAIL   USED    USEDDS
tank/usr/ports            2.50T   978M      978M
tank/usr/uncompressed     2.50T  2.95G     2.95G
```

Overview

1. Introduction
2. Problems with Today's Filesystems
3. Features of ZFS
   - Simple Administration
   - Quota and Reservations
   - Snapshots
     - Clones
     - Boot Environments
   - Self-Healing Data
   - Compression
   - Deduplication
   - ZFS Volumes
   - ZFS Serialization

Deduplication

Data is often intentionally stored multiple times on data storage devices and filesystems for reasons such as redundancy. However, there are also cases of unintentional or unnoticed duplication, like caching or temporary files. Some applications may store data in hidden working directories, leading to additional disk space usage for each stored copy.
Deduplication serves as a method to reduce this redundant data, acting as the opposite of redundancy.
ZFS implements deduplication at the block layer. When deduplication is activated on a ZFS pool, and checksums detect that a block B is an exact copy of an existing block A, a reference is stored from B to A instead. This reference takes up only a few bytes and is managed transparently for the user. This approach significantly saves disk space in situations where there is a substantial amount of duplicate data.

Using Deduplication

To activate deduplication on a pool, set it using the following command:

```
zfs set dedup=on pool
```

With this setting, ZFS compares a checksum of every new block and if there is such a block on the pool already then a references replaces the new block. These checks happen in memory, which is why not every pool should activate deduplication. Blocks that were already written and are duplicates of one another are not deduplicated after the activation. Instead, copy the data again or change it (i.e. their timestamps using touch(1)) to trigger the activation of the deduplication checks.
A pool with activated deduplication typically looks like this:

```
# zpool list
NAME       SIZE   ALLOC    FREE     CAP   DEDUP   HEALTH   ALTROOT
mypool    2.84G   2.19M   2.83G      0%   1.00x   ONLINE   -
```

The `DEDUP` column shows the current rate of deduplication.

Deduplication Example

In this example data stored in different directories on the same dataset on a pool with activated deduplication.

```
# zpool list
NAME       SIZE   ALLOC    FREE     CAP   DEDUP   HEALTH   ALTROOT
mypool    2.84G   2.19M   2.83G      0%   1.00x   ONLINE   -
# zfs get dedup mypool
NAME    PROPERTY  VALUE           SOURCE
mypool   dedup      on              local
# cd /mypool
# for d in dir1 dir2 dir3; do
for> mkdir $d && cp -R /usr/ports $d &
for> done
...
# zpool list
NAME       SIZE   ALLOC    FREE     CAP   DEDUP   HEALTH   ALTROOT
mypool    2.84G   20.9M   2.82G      0%   3.00x   ONLINE   -
# df -h mypool
Filesystem     Size     Used    Avail Capacity  Mounted on
mypool         2.8G      54M     2.8G      2%    /mypool
```

Checking whether Deduplication is Beneficial

One can check whether it would be beneficial to activate ZFS' deduplication feature for a given set of data. For that, we use the ZFS debugger called `zdb`, which has an option to simulate the deduplication benefits.

```
# zdb -S mypool
Simulated DDT histogram:

    bucket              allocated                       referenced
    ------   ------------------------------   ------------------------------
    refcnt   blocks   LSIZE   PSIZE   DSIZE   blocks   LSIZE   PSIZE   DSIZE
    ------   ------   -----   -----   -----   ------   -----   -----   -----
         1    17.6K   32.6M   32.6M   32.6M    17.6K   32.6M   32.6M   32.6M
         2      102   73.5K   73.5K   73.5K      224    164K    164K    164K
         4        6      3K      3K      3K       30     15K     15K     15K
        16        1      1K      1K      1K       23     23K     23K     23K
     Total    17.7K   32.7M   32.7M   32.7M    17.9K   32.8M   32.8M   32.8M

     dedup = 1.00, compress = 1.00, copies = 1.00, dedup * compress / copies = 1.00
```

In this example, activating deduplication *will not be beneficial*. Factors that favor deduplication are an activated compression and the usage of the `copies` property that stores copies of a block for redundancy reasons (in case of bad blocks on USB sticks, memory cards, etc.). These redundant copies are what deduplication needs.

Overview

1. Introduction
2. Problems with Today's Filesystems
3. Features of ZFS
   - Simple Administration
   - Quota and Reservations
   - Snapshots
     - Clones
     - Boot Environments
   - Self-Healing Data
   - Compression
   - Deduplication
   - ZFS Volumes
   - ZFS Serialization

Outfitting non-ZFS Filesystems with ZFS Properties

Sometimes, ZFS may not be suitable as a filesystem for a specific task, but its features can still be beneficial. In this example, we will create a FAT32 filesystem to exchange data with Windows systems using a volume. We'll export a volume as a block device, using it as free, contiguous storage. Later, we'll create another filesystem (in our case: FAT32) on this raw device. Despite FAT32 not supporting compression, we'll activate compression through ZFS to gain more disk space. Another use case for volumes is using them as swap devices. Additionally, exporting ZFS volumes as iSCSI devices over the network to other computers is possible. In this scenario, it will appear as free, *local* storage (iSCSI property).

Outfitting non-ZFS Filesystems with ZFS Properties

```
# zfs create -V 250m -o compression=on mypool/fat32
```

The volume does not show up as a separate ZFS dataset, there is no `mypool/fat32` in this output:

```
# zfs list mypool
NAME     USED  AVAIL  REFER  MOUNTPOINT
mypool   258M   670M    31K  /mypool
```

Stored in the path `/dev/zvol/mypool` instead.

```
# newfs_msdos -F32 /dev/zvol/mypool/fat32
# mount -t msdosfs /dev/zvol/mypool/fat32 /mnt
# df -h /mnt
Filesystem                Size    Used   Avail Capacity  Mounted on
/dev/zvol/mypool/fat32    249M     24k    249M     0%    /mnt
# mount
...
/dev/zvol/mypool/fat32 on /mnt (msdosfs, local)
```

The filesystem can now use a Windows shared folder without the knowledge its backed by a compressed ZFS dataset.

Overview

1. Introduction
2. Problems with Today's Filesystems
3. Features of ZFS
   - Simple Administration
   - Quota and Reservations
   - Snapshots
     - Clones
     - Boot Environments
   - Self-Healing Data
   - Compression
   - Deduplication
   - ZFS Volumes
   - ZFS Serialization

ZFS Serialization

ZFS contains built-in functionality to serialize the storage so that it sends a byte stream to standard output. For that functionality, use snapshots of the dataset. This way tranferring a dataset consistently between two systems running ZFS pools is possible. The command is `zfs send`. The following two pools help to illustrate this feature:

```
# zpool list
NAME      SIZE   ALLOC    FREE     CAP   DEDUP   HEALTH   ALTROOT
backup    960M     77K    960M      0%   1.00x   ONLINE   -
mypool    984M   43.7M    940M      4%   1.00x   ONLINE   -
```

ZFS Serialization

The following two pools help to illustrate this feature:

```
# zpool list
NAME     SIZE   ALLOC    FREE     CAP   DEDUP   HEALTH   ALTROOT
backup   960M     77K    960M      0%   1.00x   ONLINE   -
mypool   984M   43.7M    940M      4%   1.00x   ONLINE   -
# zfs snapshot mypool@backup1
# zfs list -t snapshot
NAME                        USED   AVAIL   REFER   MOUNTPOINT
mypool@backup1                 0       -   43.6M   -
# zfs send mypool@backup1
Error: Stream can not be written to a terminal.
You must redirect standard output.
# zfs send mypool@backup1 > /backup/backup1
# zpool list
NAME     SIZE   ALLOC    FREE     CAP   DEDUP   HEALTH   ALTROOT
backup   960M   63.7M    896M      6%   1.00x   ONLINE   -
mypool   984M   43.7M    940M      4%   1.00x   ONLINE   -
```

ZFS Incremental Backups

Serializing changes between two snapshots and transfer them using `zfs send` (delta replication). This forms an incremental backup solution.

```
# zfs snapshot mypool@backup2
# zfs list -t snapshot
NAME                    USED  AVAIL  REFER  MOUNTPOINT
mypool@backup1         5.72M      -  43.6M  -
mypool@backup2             0      -  44.1M  -
# zpool list
NAME     SIZE  ALLOC   FREE    CAP  DEDUP  HEALTH  ALTROOT
backup   960M  61.7M   898M     6%  1.00x  ONLINE  -
mypool   960M  50.2M   910M     5%  1.00x  ONLINE  -
# zfs send -i mypool@backup1 mypool@backup2 > /backup/diff
# zpool list
NAME     SIZE  ALLOC   FREE    CAP  DEDUP  HEALTH  ALTROOT
backup   960M  80.8M   879M     8%  1.00x  ONLINE  -
mypool   960M  50.2M   910M     5%  1.00x  ONLINE  -
# ls -lah /backup
total 82247
drwxr-xr-x   2 root  wheel     4B Dec  3 11:46 .
drwxr-xr-x  21 root  wheel    28B Dec  3 11:32 ..
-rw-r--r--   1 root  wheel    61M Dec  3 11:36 backup1
-rw-r--r--   1 root  wheel    18M Dec  3 11:46 diff
```

ZFS Incremental Backups - Restore Data

We received the backups, but they are still stored in the binary stream format. When we want to get to the data itself, we need to use `zfs send` in conjunction with `zfs receive` (or the abbreviated `zfs recv`).

```
# zfs send mypool@backup1 | zfs receive backup/backup1
# ls -lah /backup/
total 431
drwxr-xr-x     3 root   wheel      3B Dec  3 12:06 .
drwxr-xr-x    21 root   wheel     28B Dec  3 12:05 ..
drwxr-xr-x  4219 root   wheel    4.1k Dec  3 11:34 backup1
```

In backup1 we will now find all the data from the snapshot `mypool@backup1`.

```
# zfs list
NAME                                 USED   AVAIL   REFER   MOUNTPOINT
backup                              43.7M    884M     32K   /backup
backup/backup1                      43.5M    884M   43.5M   /backup/backup1
mypool                              50.0M    878M   44.1M   /mypool
```

ZFS Backups – Encrypted Backups over the Network

Since ZFS works with standard out, use the known functionalities like redirections on the output. Encrypt the streams locally or in transit via SSH to another cold standby server over an insecure network like the Internet. To make this work, these conditions must apply:

- passwordless SSH access using public keys to the target host
- `root` (or another privileged user) must be able to log in via SSH
- the target system must allow `root` the execution of `zfs recv` (configurable in SSH)

Schedule a cron(8) job to perform the backup in regular intervals.

ZFS Backups – Encrypted Backups over the Network

For our example, we perform a recursive backup of all home directories on `host1` and send them to another host called `host2`.

```
host1# zfs snapshot -r tank/home@monday
host1# zfs send -R tank/home@monday | ssh host2 zfs recv -dvu pool
```

The option `-R` takes care of recursively descending into all child datasets. This includes snapshots, clones, and property settings.
The option `-d` in `zfs recv` will remove the original pool name on the receiving side and use the name of the snapshot. The `-u` option determines that the receiving dataset should not mount. Display more information using `-v`, for example the elapsed time during the receiving operation.

Further Information



[FBSDHBZFS] The FreeBSD Handbook - 21.2 The Z File System (ZFS)
http://www.freebsd.org/doc/en_US.ISO8859-1/books/handbook/zfs.html



[ZFSLinux] ZFS on Linux
http://zfsonlinux.org



[KLARABEMGMT] Klara Systems Blog: Managing Boot Environments
https://klarasystems.com/articles/managing-boot-environments/



[CSIMUNICH] CSI:Munich - Demonstrating ZFS on USB sticks
http://www.youtube.com/watch?v=1zw8V8g5eT0



[ZFSBE] Slawomir Wojciech Wojtczak: ZFS Boot Environments, Polish BSD User Group 2018/07/30
https://bsd-pl.org/assets/talks/2018-07-30_1_S%C5%82awomir-Wojciech-Wojtczak_ZFS-Boot-Environments.pdf